



The Shed, explained

A file and folder browser on the starter you sent, built the way I build my own project: shape and tests first, every decision written down, and a live site.

MAPLARGE DEVELOPER TEST PROJECT | STEVEN STOUT | 29 SEPTEMBER 2026

LIVE theshed.stevenstout.biz

CODE github.com/SteveStout/TheYard, [samples/maplarge](#)

RESUME AND ABOUT on the site, one click each

What it is

One configurable home directory, one API, one page. The address bar is the whole state.

An ASP.NET Core 10 API over a home directory set by configuration, and a page in TypeScript with no framework. It browses, searches by name or wildcard, uploads with a progress bar, downloads, makes folders, moves, copies and deletes, inside a native dialog with one trigger. Every failure is a sentence the page shows. Twelve decision records are served from the running app, each with the code it decided about read from the build as you read it.

QUICK START

```
dotnet run # localhost:5120, browsing sample-home/
dotnet test # 112 xUnit tests, warnings as errors
npm install && npm test # 21 node tests over the compiled modules
```

OR SKIP THE BUILD, THREE LINKS

- theshed.stevenstout.biz/?path=reports/2026: the dialog, open in a folder.
- [/?q=*.csv&sort=size&dir=desc](https://theshed.stevenstout.biz/?q=*.csv&sort=size&dir=desc): a search, sorted, as a link.
- [/?view=docs&doc=adr-003-the-line-a-path-cannot-cross](https://theshed.stevenstout.biz/?view=docs&doc=adr-003-the-line-a-path-cannot-cross): the security record, its code live. Read ADR-003, 005 and 009 first; the README is the map.

The brief, and where each item is

You asked for	Where it is (paths under samples/maplarge)
Builds in VS, Rider, VS Code, CLI	TestProject.sln, your file kept by name, on .NET 10; no package.
JSON API: browse and search	Controllers/FilesController.cs; every failure an RFC 9457 document from Controllers/ProblemResponseHandler.cs.
Home directory by variable	Files:Home in appsettings.json or FILES__HOME; Domain/HomePath.cs guards every path.
Deep-linkable state in the URL	src/lib/urlState.ts parses and writes six keys; src/main.ts has the one navigate().
Vanilla SPA, client rendered	TypeScript in src/, compiled by tsc to wwwroot/js; src/lib is pure, src/ui renders.
Upload and download	src/lib/api.ts uploads with a bar per file; downloads stream with ranges.
Counts and sizes per view	Domain/ViewTotals.cs, on every reply from Application/FileBrowser.cs, plus took_ms.
Bonus: dialog, writes, performance	A native dialog in wwwroot/index.html; writes ask in the row; tests/TestProject.Tests/PerformanceTests.cs.

How it was built and shipped

- **Green before every commit.** 133 tests, warnings as errors, and a headless Chrome pass of the running page (deep links, search, Back, upload, delete, Escape) with zero console errors.
- **Live at no added cost.** A third web app on the App Service plan my project runs on, its own free managed certificate, the image in the same registry, rolled by a workflow whose identity holds Website Contributor on that one site and nothing wider. The Bicep and the workflow are in the folder.
- **Built with AI, and written down (ADR-011).** I decided the shape and the test list first, Claude drafted against them, the build ran on my machine, and I read everything before it shipped. My first push had a folder-name case collision Windows hides and Linux does not; the container found it, and the link test now checks case exactly.

By the numbers

133 tests, green before every commit	< 1 ms to browse 100 files, measured by a test	47 ms to search 10,000 files whole; capped, 6 ms
0 warnings, analyzers at their strictest	12 records served live with their code	\$0 added hosting, on a plan I already pay for

Over 10,000 files on the build machine (ADR-008)

lower is better

Browse, 100 files		< 1 ms
Search, capped at 200		6 ms
Search, whole tree		47 ms

Six choices, and why

1 Your starter, kept Your names, your controller shape, your one page. Three lines added to the project file (ADR-001).	2 Dependencies inward One project, four folders. A test reads every using and fails on one that points the wrong way (ADR-002).
3 One pure security class Three string refusals before any filesystem touch, then containment; the same tests on both OSes (ADR-003).	4 The address is the state Nothing is kept anywhere else. The first keystroke of a search pushes; the rest replace (ADR-005).
5 Measured, not claimed Search is lazy, stops at a cap and says so; a test prints the timings on every run (ADR-008).	6 Rules beside their tests A table of every standing rule and the test that holds it; a test reads the table (ADR-009).

Taking it further: my larger project is the worked example

TheYard (theyard.stevenstout.biz) is the same practice at full size: .NET 10 and React, Azure SQL and Cosmos DB, 86 records, about 1,950 test runs per gate, two sites on one plan. Each step below is something the Yard already does.

Next on the Shed	How the Yard already does it
A committed browser test	143 Playwright specs in the gate, located by role and name; axe holds nine views to WCAG 2.1 AA.
Paging behind the search cap	The inventory API pages 100,000 vehicles behind Load more; the page never asks for the whole set.
Logs that outlive the container	Every request and error kept in Cosmos DB for three years, read on the Admin tab (ADR-071, 073).
The path guard as a Files card	The Admin tab is already a product: health, machines, timing, kept logs, each a card (ADR-080).

1. Program.cs is a table of contents (ADR-002)

The first file a reviewer opens answers one question: what is this app made of, in what order. Each line names a part, and the file beside it shows how. No rule lives here, so there is nothing to review but the order.

samples/maplarge/Program.cs

```

/// <summary>
/// The entry point. It builds the web host and starts it. Each step is a single call, and the comment beside the call names the file that holds the
/// details. The setup lives in those files so that this class stays a short, readable list of what the app is made of.
/// </summary>
public sealed class Program
{
    /// <summary>Builds the web app from its parts, in order, and runs it until shutdown.</summary>
    /// <param name="args">Command line arguments, passed to the host builder.</param>
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        builder.AddTheShedFiles(); // Composition/FilesRegistration.cs
        builder.AddTheShedDocumentationAndVersion(); // Composition/DocumentationAndVersionRegistration.cs
        builder.AddTheShedApi(); // Composition/ApiRegistration.cs

        var app = builder.Build();
        app.UseTheShedRequestPipeline(); // Composition/RequestPipeline.cs
        app.MapControllers(); // Controllers/: Files, DocumentationAndVersion, Health

        app.Run();
    }
}

```

2. The file browser, registered (ADR-002, ADR-003)

The first call in Program.cs. The home folder comes from configuration, the rules depend only on the IFileStore interface, and the upload limit is set so an oversized file gets a message that names it. The disk keeps the sample simple; production would use Azure Blob Storage or another cloud file store.

samples/maplarge/Composition/FilesRegistration.cs, AddTheShedFiles

```

/// <summary>
/// Registers the file browser's services: the home directory and its limits, the file store interface with the class that reads and writes real
/// files, and the FileBrowser that holds the rules. It also sets the server's upload size limits to match the configured maximum.
/// </summary>
public static class FilesRegistration
{
    /// <summary>
    /// Adds the home directory, the file store, the FileBrowser and the upload size limits to the host.
    /// </summary>
    /// <param name="builder">The host being built.</param>
    public static void AddTheShedFiles(this WebApplicationBuilder builder)
    {
        // Reads the "Files" section of configuration (or FILES__* environment variables) into FilesOptions, then registers the options object and the
        // HomePath built from it as singletons, so every class receives the same home directory.
        builder.Services.Configure<FilesOptions>(builder.Configuration.GetSection(FilesOptions.Section));
        builder.Services.AddSingleton(provider => provider.GetRequiredService<IOptions<FilesOptions>>().Value);
        builder.Services.AddSingleton(provider => HomeFor(provider.GetRequiredService<FilesOptions>()), builder.Environment.ContentRootPath));

        // IFileStore is the interface the rules use to touch the disk, and PhysicalFileStore is the class that reads and writes real files.
        // FileBrowser holds the rules and depends only on the interface, so tests can swap in a different store.
        //
        // This sample keeps files on the server's own disk to stay simple. For production, store them in Azure Blob Storage or another cloud file
        // storage service, the way TheYard keeps its data in managed Azure services rather than on a server. A container's disk is wiped when the app
        // restarts or redeploys and is not shared between instances, so uploads here do not last. The move is one new class that implements
        // IFileStore and this one line.
        builder.Services.AddSingleton<IFileStore, PhysicalFileStore>();
        builder.Services.AddSingleton<FileBrowser>();

        // FileBrowser.UploadAsync checks the exact per-file limit and names the file in its error. The web server and the form parser get that limit
        // plus one megabyte, to allow for the multipart form's own headers and boundaries. A slightly large request therefore still reaches the
        // per-file check and gets a clear message, while a far too large request is cut off by the server before it is read.
        long maxUpload = builder.Configuration.GetSection(FilesOptions.Section).GetValue("MaxUploadBytes", new FilesOptions().MaxUploadBytes);
        long slack = 1024 * 1024;
        builder.WebHost.ConfigureKestrel(kestrel => kestrel.Limits.MaxRequestBodySize = maxUpload + slack);
        builder.Services.Configure<FormOptions>(form => form.MultipartBodyLengthLimit = maxUpload + slack);
    }
    ...
}

```

3. The documents and the version, registered (ADR-012)

The second call in Program.cs. It sets nothing itself: the version comes from the top of docs/CHANGELOG.md, the commit from the deploy, and the documents from the docs folder. This file makes their readers available to the controllers.

samples/maplarge/Composition/DocumentationAndVersionRegistration.cs

```
/// <summary>
/// Registers the two services behind the Docs tab and the version in the page footer. This file does not set the version or write any documentation.
/// It only makes the readers available to the controllers:
/// <list type="bullet">
/// <item>The version number is set by the newest entry at the top of docs/CHANGELOG.md.
/// Adding a line there is how a new version is set.</item>
/// <item>The commit is set by the deploy, which puts it in the SHED_COMMIT environment
/// variable of the container. A local run reads it from the .git folder instead.</item>
/// <item>The documentation is the markdown files in the docs folder (the decision records
/// and the guides) plus README.md. DocumentationAndVersionController serves them to the Docs tab.</item>
/// </list>
/// </summary>
public static class DocumentationAndVersionRegistration
{
    /// <summary>
    /// Adds DocumentationCatalog, which lists the documents in sidebar order and finds one by name, reading them from disk on every request so an
    /// edited file shows on the next page load. Adds VersionInfo, which VersionReader reads once at startup from the changelog and the commit sources
    /// above. Both are singletons rooted at the content root.
    /// </summary>
    /// <param name="builder">The host being built.</param>
    public static void AddTheShedDocumentationAndVersion(this WebApplicationBuilder builder)
    {
        builder.Services.AddSingleton(new DocumentationCatalog(builder.Environment.ContentRootPath));
        builder.Services.AddSingleton(VersionReader.Read(builder.Environment.ContentRootPath));
    }
}
```

4. The line a path cannot cross (ADR-003)

Empty and rooted cases first, then two refusals as strings, then the joined path must still start inside the root. Pure, so the same tests run on Windows and Linux.

samples/maplarge/Domain/HomePath.cs, Resolve

```
/// <summary>
/// Turns a path from a request into an absolute path under the root, or throws <see cref="PathRefusedException"/>. It first refuses three things by
/// looking at the string alone: a rooted path (a drive letter or a leading slash), a "." or ".." segment anywhere, and a segment containing a
/// character the operating system does not allow in a name. It then joins the segments onto the root, normalises the result, and checks it still lies
/// inside the root. That final check is the safety net for any trick the first three checks do not anticipate.
/// </summary>
/// <param name="relative">The path the request sent; null or "" means home itself.</param>
public string Resolve(string? relative)
{
    string trimmed = (relative ?? string.Empty).Trim();
    if (trimmed.Length == 0)
    {
        return Root;
    }
    if (System.IO.Path.IsPathRooted(trimmed) || trimmed.StartsWith('/') || trimmed.StartsWith('\\'))
    {
        throw new PathRefusedException("A path must be relative to the home directory.");
    }
    string[] segments = trimmed.Split(Separators, StringSplitOptions.RemoveEmptyEntries);
    foreach (string segment in segments)
    {
        if (segment is "." or "..")
        {
            throw new PathRefusedException("A path cannot contain '.' or '..' segments.");
        }
        if (segment.IndexOfAny(System.IO.Path.GetInvalidFileNameChars()) >= 0)
        {
            throw new PathRefusedException($"'{segment}' is not a valid name.");
        }
    }
    string joined = System.IO.Path.GetFullPath(System.IO.Path.Combine(Root, System.IO.Path.Combine(segments)));
    if (!IsInside(joined))
    {
        throw new PathRefusedException("A path must stay inside the home directory.");
    }
    return joined;
}
```

5. The tests that hold it

Every refusal is a case, and the prefix case is the one most guards miss: /home/files-old is not inside /home/files.

`samples/maplarge/tests/TestProject.Tests/HomePathTests.cs`

```
/// <summary>
/// Checks HomePath, the guard that keeps every requested path inside the home folder. The tests pass plain strings and never touch a disk, with a
/// root in the form of whichever OS runs them. This guard is what stops a request from reading or writing outside the home, so each way of escaping
/// it (dot segments, rooted paths, bad characters, shared prefixes) has its own case. (more in docs/ADR-003-the-line-a-path-cannot-cross.md)
/// </summary>
public sealed class HomePathTests
{
    ...
    [InlineData("../secrets")]
    [InlineData("docs/../../secrets")]
    [InlineData("docs/./notes")]
    [InlineData("..")]
    public void A_dot_or_dot_dot_segment_is_refused(string path)
    {
        var refused = Assert.Throws<PathRefusedException>(() => _home.Resolve(path));
        Assert.Contains("'.' or '..'", refused.Message, StringComparison.Ordinal);
    }
    ...
    public void The_root_with_a_prefix_in_common_is_outside()
    {
        // /home/files-old starts with the same characters as /home/files but is a different folder. A plain string prefix check would wrongly call it
        // inside.
        Assert.False(_home.IsInside(_home.Root + "-old"));
        Assert.False(_home.IsInside(Path.Combine(_home.Root + "-old", "x")));
    }
}
```

6. Every failure is a sentence (ADR-004)

One handler maps each refusal to a status and writes an RFC 9457 problem document with a trace id. The controllers have no try/catch.

`samples/maplarge/Controllers/ProblemResponseHandler.cs`

```
/// <summary>
/// Turns a refused request into an RFC 9457 problem document. A <see cref="ApiRefusalException"/> carries its own status and title. A <see
/// cref="PathRefusedException"/> from the home directory check is always a 400. Refusals from the filesystem itself, such as a read-only file inside
/// a folder being deleted or a file another process holds open, become a 403 or a 409 carrying the operating system's message. They are treated as
/// client errors because the request asked for something the disk cannot do, not because the server is broken. Any other exception is left unhandled
/// here, so a real bug still reaches the default handler as a 500 with a trace id and is never reported as a client error.
/// </summary>
public sealed class ProblemResponseHandler(IProblemDetailsService problems) : IExceptionHandler
{
    /// <inheritdoc />
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, CancellationToken cancellationToken)
    {
        (int status, string title) = exception switch
        {
            ApiRefusalException problem => (problem.Status, problem.Title),
            PathRefusedException => (StatusCodes.Status400BadRequest, "The path was refused"),
            UnauthorizedAccessException => (StatusCodes.Status403Forbidden, "The filesystem refused"),
            IOException => (StatusCodes.Status409Conflict, "The filesystem refused"),
            _ => (0, string.Empty),
        };
        if (status == 0)
        {
            return false;
        }
        httpContext.Response.StatusCode = status;
        return await problems.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = httpContext,
            ProblemDetails = new ProblemDetails
            {
                Status = status,
                Title = title,
                Detail = exception.Message,
                // Adds the same trace id the logs carry, so a reported failure can be found there.
                Extensions = { ["TraceId"] = Activity.Current?.Id ?? httpContext.TraceIdentifier },
            }
        });
    }
}
```

7. A search that stops (ADR-008)

The walk is lazy and the cap is clamped, so a search that fills up in the first subfolder never reads the rest of the tree, and the reply says truncated.

`samples/maplarge/Application/FileBrowser.cs`, `Search`

```

/// <summary>
/// Finds every file and folder under a folder, at any depth, whose name matches the query, and stops once it has the maximum number of matches. The
/// store hands back entries one at a time as it walks, so a search that fills up early never reads the rest of the tree. That early stop and the cap
/// on results keep a search on a large tree fast (more in docs/ADR-008-performance-measured.md).
/// </summary>
/// <param name="path">The folder to search under, relative to home; "" means home.</param>
/// <param name="query">A plain substring, or a glob using * or ?; empty is refused.</param>
/// <param name="limit">
/// The most matches to return. Null uses the configured default, and any value is clamped between 1 and the configured ceiling. </param>
public SearchResult Search(string? path, string? query, int? limit)
{
    long started = Stopwatch.GetTimestamp();
    var pattern = new NamePattern(query);
    if (pattern.IsEmpty)
    {
        throw ApiRefusalException.Refused("A search needs something to look for.");
    }
    int cap = Math.Clamp(limit ?? options.SearchLimit, 1, options.SearchCeiling);
    string absolute = Folder(path);
    var folders = new List<FolderEntry>();
    var files = new List<FileEntry>();
    bool truncated = false;
    foreach (StoreEntry entry in store.Descendants(absolute))
    {
        if (!pattern.Matches(Path.GetFileName(entry.Absolute)))
        {
            continue;
        }
        if (folders.Count + files.Count >= cap)
        {
            truncated = true;
            break;
        }
        Sort(entry, folders, files);
    }
    folders.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
    files.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
    return new SearchResult(pattern.Query, Home.Relative(absolute), folders, files, ViewTotals.Of(folders, files), truncated, Elapsed(started));
}

```

8. The address is the state (ADR-005)

One parser and one serializer. A value outside its list falls back, a default is never written, so a link is as short as it can be.

`samples/maplarge/src/lib/urlState.ts`

```

/**
 * Reads a query string into a State. Unknown keys are ignored. A value that is not in its allowed list falls back to the default. A path is cleaned
 * to forward slashes with no leading or trailing slash, so two ways of writing the same folder give the same State.
 * @param search the query string from the address, with or without the leading "?"
 */
export function parse(search: string): State {
    const params = new URLSearchParams(search.startsWith('?') ? search.slice(1) : search);
    return {
        // Any known key opens the dialog, so a link as short as "?path=docs" works. An open dialog with every other value at its default is written as
        // "?view=browse" by serialize().
        open: KEYS.some((key) => params.has(key)),
        view: pick(params.get('view'), VIEWS, DEFAULTS.view),
        path: cleanPath(params.get('path')),
        q: (params.get('q') ?? '').trim(),
        sort: pick(params.get('sort'), SORTS, DEFAULTS.sort),
        dir: pick(params.get('dir'), DIRS, DEFAULTS.dir),
        doc: (params.get('doc') ?? '').trim().toLowerCase(),
    };
}

/**
 * Writes a State as a query string. Values equal to their default are left out, so links stay as short as possible. A closed State gives "" (no
 * query at all). An open State where every value is the default would also give nothing, so it writes "?view=browse" to keep the dialog open when
 * the address is read back. @returns "" when closed, otherwise "?key=value&..."
 */
export function serialize(state: Partial<State>): string {
    const full: State = { ...DEFAULTS, open: true, ...state, path: cleanPath(state.path) };
    if (!full.open) {
        return '';
    }
    const params = new URLSearchParams();
    for (const key of KEYS) {
        if (full[key] !== DEFAULTS[key] && full[key] !== '') {
            params.set(key, full[key]);
        }
    }
    if ([...params.keys()].length) {
        params.set('view', full.view);
    }
    return `?${params.toString()}`;
}

```

9. The one way the state changes (ADR-005)

Merge, write the address, redraw. Closing is navigating to the defaults, so Back from a closed page reopens it where it was.

`samples/maplarge/src/main.ts, navigate`

```
/**
 * Changes the page state. It merges the changed keys into the current state, writes the result into the address bar as a history entry, then
 * redraws. Every state change in the app comes through here. Closing the dialog is also a navigation (to the default state), so pressing Back after
 * closing reopens the dialog where it was.
 * @param partial the state keys that change; keys left out keep their current value
 * @param replaceEntry true to overwrite the current history entry instead of adding one
 * (used while typing in the search box, so each keystroke is not its own Back step)
 */
function navigate(partial: Partial<State>, replaceEntry = false): void {
  const next: State = { ...state, open: true, ...partial };
  // A closed state serializes to an empty string. pushState with "" would keep the old query string, so the page path with no query string is written
  // instead, which clears it.
  const url = serialize(next) || window.location.pathname;
  if (replaceEntry) {
    window.history.replaceState(null, '', url);
  } else {
    window.history.pushState(null, '', url);
  }
  // Parse the address again instead of keeping `next`. The address is the only source of truth, and parsing it also cleans up any value the
  // serializer left out or would have rejected.
  state = parse(window.location.search);
  render();
}
```